

Dokumentasi Sistem Chattingan

System Flow & Architecture Documentation

Versi: 1.0

Tanggal: 13 Juni 2026

Platform: Web (Next.js + Express + Socket.IO + MySQL)

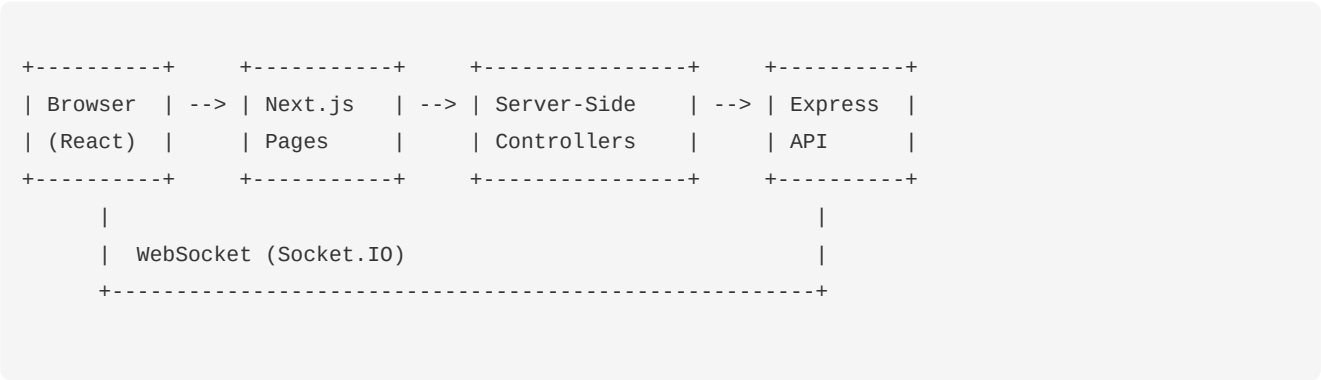
Daftar Isi

1. Arsitektur Sistem.....	3
1.1 High-Level Architecture.....	3
1.2 Struktur Folder Frontend.....	4
1.3 Struktur Folder Backend.....	5
2. Alur Data.....	6
2.1 Alur Request (Proxy Pattern).....	6
2.2 Alur Autentikasi.....	7
3. Real-time Chat Flow (WebSocket).....	9
3.1 Koneksi WebSocket.....	9
3.2 Kirim & Terima Pesan.....	10
4. Database Schema.....	11
5. Detail Teknis.....	12
5.1 Authentication Strategy.....	12
5.2 Error Handling Flow.....	12
5.3 Environment Variables.....	13
6. Alur Navigasi Halaman.....	13
7. Alur Inisialisasi Aplikasi.....	14
8. Format Response API.....	15
9. Flow Diagram (Textual).....	16

1. Arsitektur Sistem

1.1 High-Level Architecture

Sistem Chattingan menggunakan arsitektur Proxy Pattern, di mana browser tidak langsung terhubung ke Backend API. Semua request melewati Next.js API Routes yang bertindak sebagai proxy layer.



Komponen utama sistem:

- Frontend (FE): Next.js Pages Router + React + Ant Design
- Backend (BE): Node.js + Express + Socket.IO + Sequelize ORM
- Database: MySQL (localhost)
- Real-time: Socket.IO (WebSocket dengan JWT auth)

1.2 Struktur Folder Frontend

```
chattingan/
  pages/
    _app.js           # App wrapper (AuthProvider, ChatProvider)
    _document.js      # Custom Document
    index.js          # Halaman daftar chat (home)
    login.js          # Halaman login
    register.js       # Halaman registrasi
    chat/[id].js      # Halaman chat room
  api/
    auth/
      login.js        # Proxy login ke BE
      register.js     # Proxy register ke BE
      me.js           # Proxy get current user
    messages/
      users.js        # Proxy get daftar user
      [userId].js     # Proxy get/send messages
  src/
    contexts/
      AuthContext.js  # Auth state (useReducer)
      ChatContext.js  # Chat state (useReducer)
    controllers/
      auth/
        login.js      # Logic server-side login
        register.js   # Logic server-side register
        me.js         # Logic server-side get user
      messages/
        get-users.js  # Logic get daftar user
        get-messages.js # Logic get riwayat chat
  lib/
    api.js            # Axios instance
    socket.js         # Socket.IO client singleton
  components/
    commons/layouts/
      MainLayout.js   # Layout utama
      MainLayout.module.css
  configurations/
    index.js          # Konfigurasi environment
```

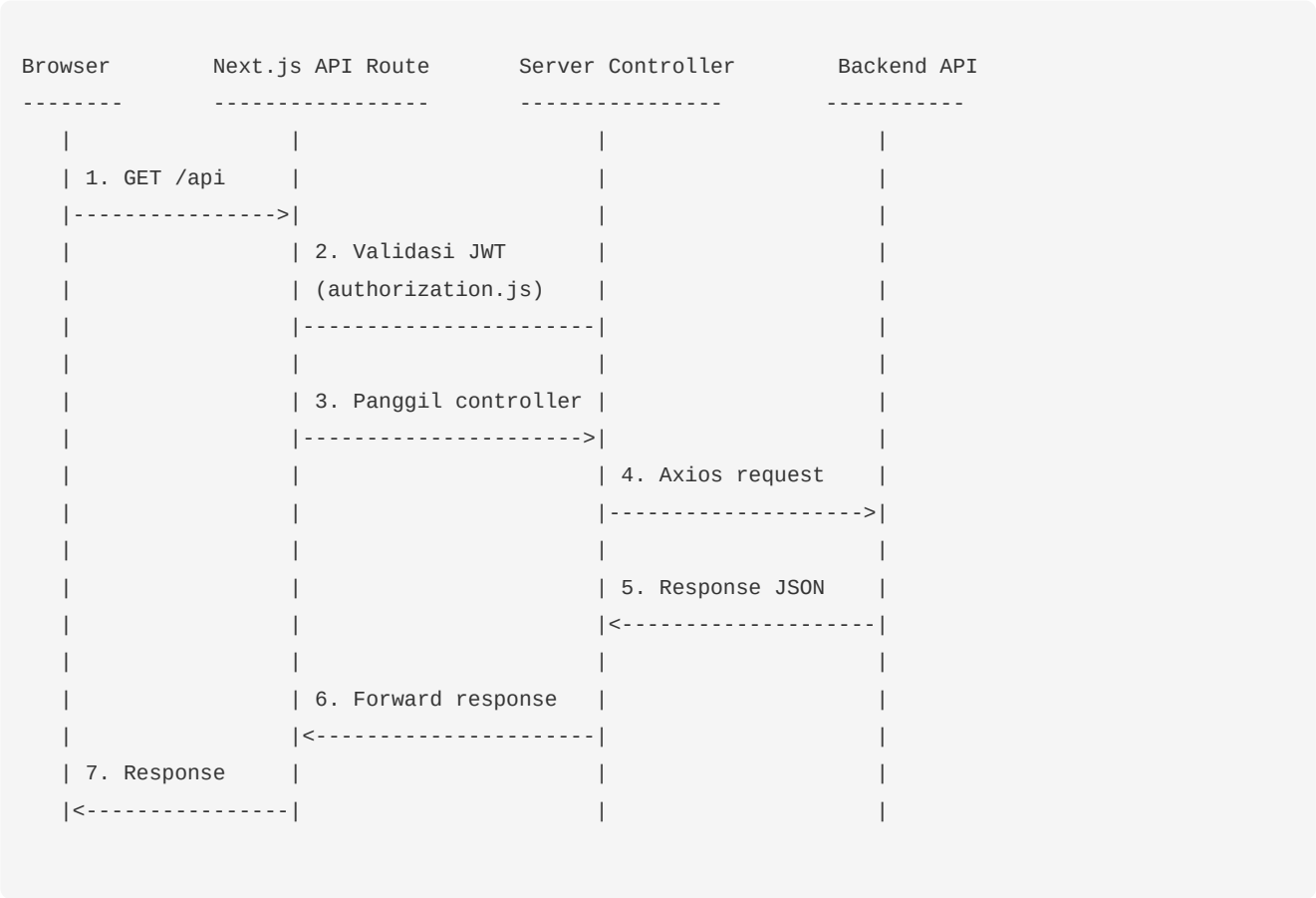
1.3 Struktur Folder Backend

```
chattingan-api/  
  src/  
    chattingan.js      # Entry point + WebSocket server + DB sync  
    app.js             # Express app setup (CORS, parser, routes)  
    configurations/  
      logger.js        # Winston logger  
    databases/  
      connections/  
        sequelize.js   # Koneksi Sequelize ke MySQL  
      models/  
        users.js       # Model User (Sequelize)  
        messages.js    # Model Message (Sequelize)  
    middlewares/  
      auth.js          # JWT authentication middleware  
    routes/  
      v1/  
        auth.js        # Route: register, login, me, logout  
        messages.js    # Route: get users, get messages  
    controllers/  
      v1/  
        auth/  
          register.js  # Registrasi user baru  
          login.js     # Login + generate JWT  
          me.js        # Get data user saat ini  
          logout.js    # Hapus cookie/session  
        messages/  
          get-users.js  # Daftar user (kecuali diri sendiri)  
          get-messages.js # Riwayat chat antara 2 user  
    functions/  
      error-handlers.js # Centralized error handler
```

2. Alur Data

2.1 Alur Request (Proxy Pattern)

Setiap request dari browser melalui 4 lapisan sebelum mencapai Backend API:



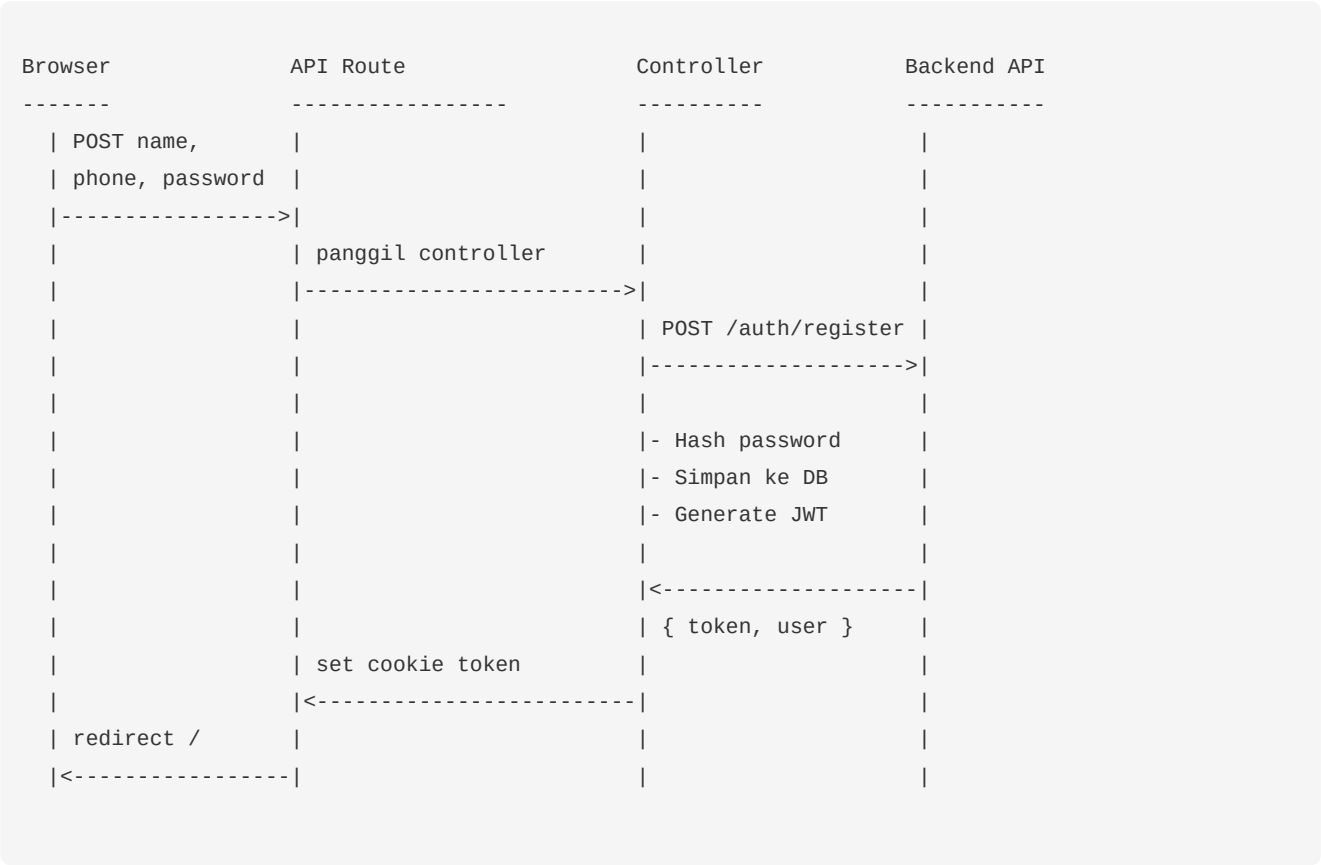
Keuntungan Proxy Pattern:

- Keamanan: Token JWT tidak terekspos ke browser
- Validasi: Semua request terautentikasi sebelum mencapai BE
- Logging: Centralized logging di API Route
- Flexibility: Bisa transform response sebelum dikirim ke browser

2.2 Alur Autentikasi

Registrasi User

User mendaftar dengan nama, nomor telepon, dan password. Backend memproses registrasi dan langsung login.



Login User

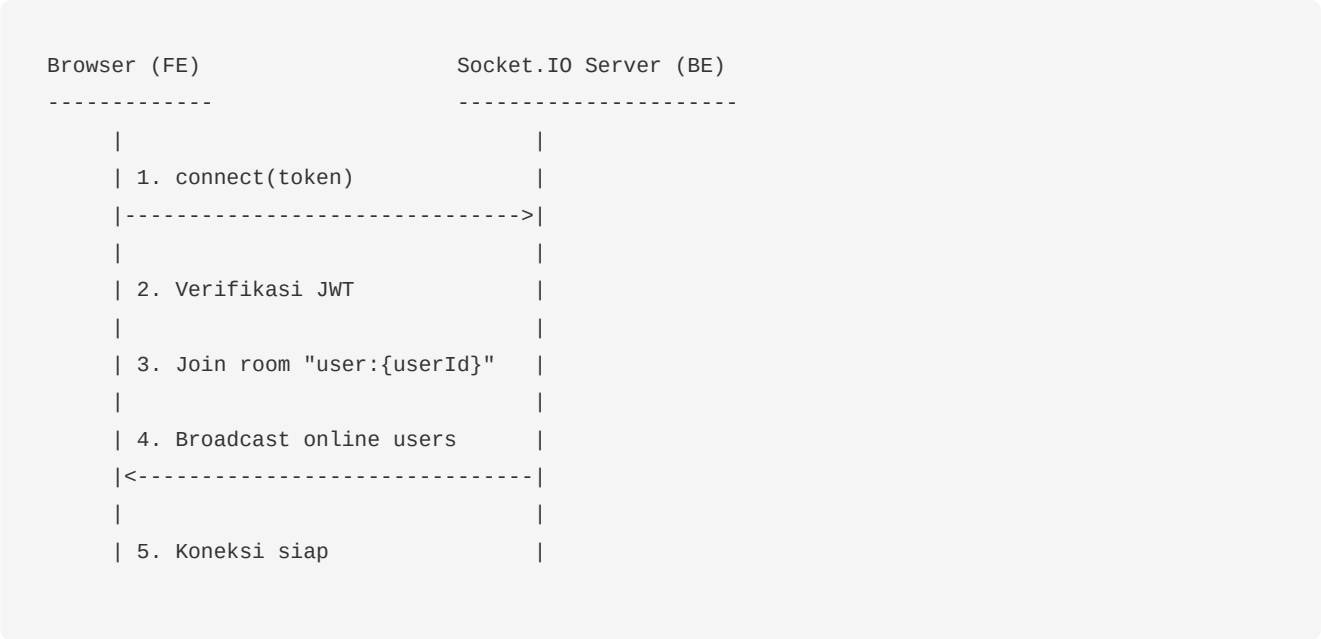
User login dengan nomor telepon dan password. Backend memvalidasi dan mengembalikan JWT token.



3. Real-time Chat Flow (WebSocket)

3.1 Koneksi WebSocket

Saat user login dan masuk ke halaman chat, FE melakukan koneksi WebSocket ke BE dengan menyertakan JWT token sebagai autentikasi.

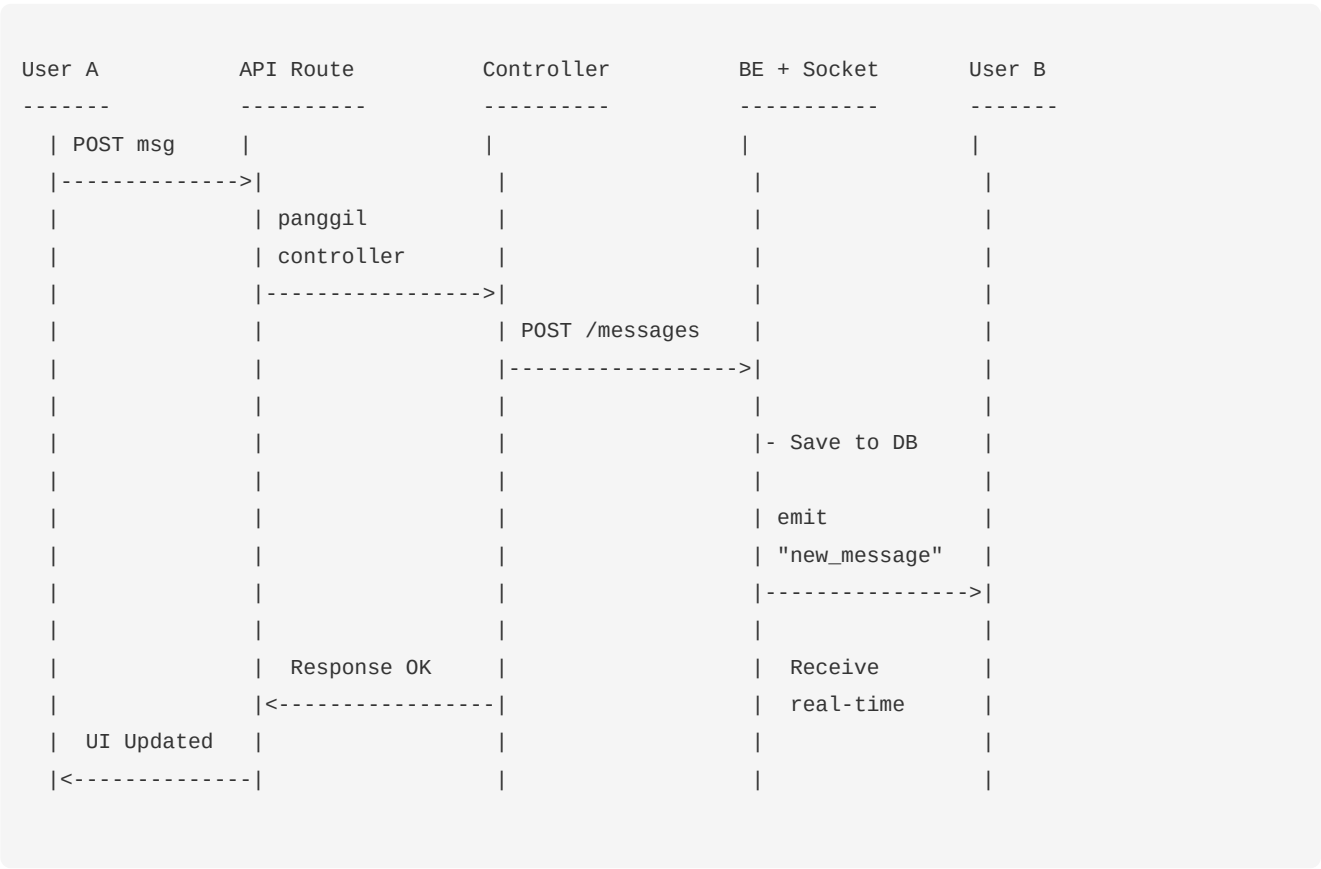


Event Socket.IO yang digunakan:

Event	Arah	Deskripsi
connect	Client -> Server	Koneksi WebSocket dengan JWT
new_message	Server -> Client	Pesan baru diterima
online_users	Server -> Client	Daftar user online diperbarui
disconnect	Client -> Server	User keluar / koneksi putus

3.2 Kirim & Terima Pesan

Flow pengiriman pesan secara real-time antara dua user:



Tahapan pengiriman pesan:

- User A mengetik pesan dan menekan Enter / Tombol Kirim
- FE mengirim POST request ke API Route /api/messages/[userId]
- API Route memvalidasi JWT dan memanggil controller
- Controller melakukan POST ke Backend API dengan Axios
- Backend menyimpan pesan ke database MySQL
- Backend mengirim event 'new_message' via Socket.IO ke User B
- User B menerima pesan secara real-time tanpa refresh
- FE User A juga memperbarui UI dengan pesan yang baru dikirim

4. Database Schema

Sistem menggunakan dua tabel utama: users dan messages.

Table: users

Column	Type	Description
id	INT (PK, AUTO_INCREMENT)	Primary key
name	VARCHAR(255)	Nama lengkap user
phone	VARCHAR(20) (UNIQUE)	Nomor telepon (login ID)
password	VARCHAR(255)	Password (bcrypt hash)
created_at	DATETIME	Waktu dibuat
updated_at	DATETIME	Waktu diupdate

Table: messages

Column	Type	Description
id	INT (PK, AUTO_INCREMENT)	Primary key
sender_id	INT (FK -> users.id)	Pengirim pesan
receiver_id	INT (FK -> users.id)	Penerima pesan
message	TEXT	Isi pesan
created_at	DATETIME	Waktu dikirim
updated_at	DATETIME	Waktu diupdate

Entity Relationship

Relasi antara users dan messages adalah one-to-many: satu user dapat mengirim banyak pesan, dan satu user dapat menerima banyak pesan.

```

+-----+      +-----+
|   users   |      |   messages   |
+-----+      +-----+
| id (PK)    |<-----| sender_id    |
| name       |      | receiver_id   |
| phone (UNIQUE) |    | message (TEXT) |
| password (hash) |   | created_at   |
| created_at |      | updated_at    |
| updated_at |      +-----+
+-----+          |      ^
                   |      |
                   +-----+
                   (FK -> users.id)

```

5. Detail Teknis

5.1 Authentication Strategy

Sistem menggunakan JWT (JSON Web Token) yang disimpan di cookie HTTP untuk autentikasi.

- JWT token dibuat saat login/register sukses
- Token disimpan di cookie (bukan localStorage) untuk keamanan lebih
- Setiap API Route memvalidasi cookie via authorization.js middleware
- Jika token invalid/expired, return 401 Unauthorized
- Logout: cookie dihapus dari response
- JWT Secret: chattingan_secret_key_2024 (dari environment variable)

5.2 Error Handling Flow

Error handling terpusat di error-handlers.js yang menangani berbagai jenis error:

Controller Logic

```
|
|--- try: execute Axios request
|   |
|   |--- success: return response 200
|   |
|   |--- error: throw to error handler
|       |
|       |--- ECONNREFUSED
|       |   -> "Service Unavailable"
|       |   -> Log warning
|       |
|       |--- error.response exists
|       |   -> Forward error dari internal service
|       |   -> Include status_code & message
|       |
|       |--- Unknown error
|       |   -> Generic error message
|       |   -> Log full error
|
|--- return response dengan format standar
```

5.3 Environment Variables

Frontend (.env.local)

Variable	Value	Description
NEXT_PUBLIC_API_URL	http://localhost:3001	Base URL Backend API
NEXT_PUBLIC_SOCKET_URL	http://localhost:3001	URL Socket.IO server

Backend (.env)

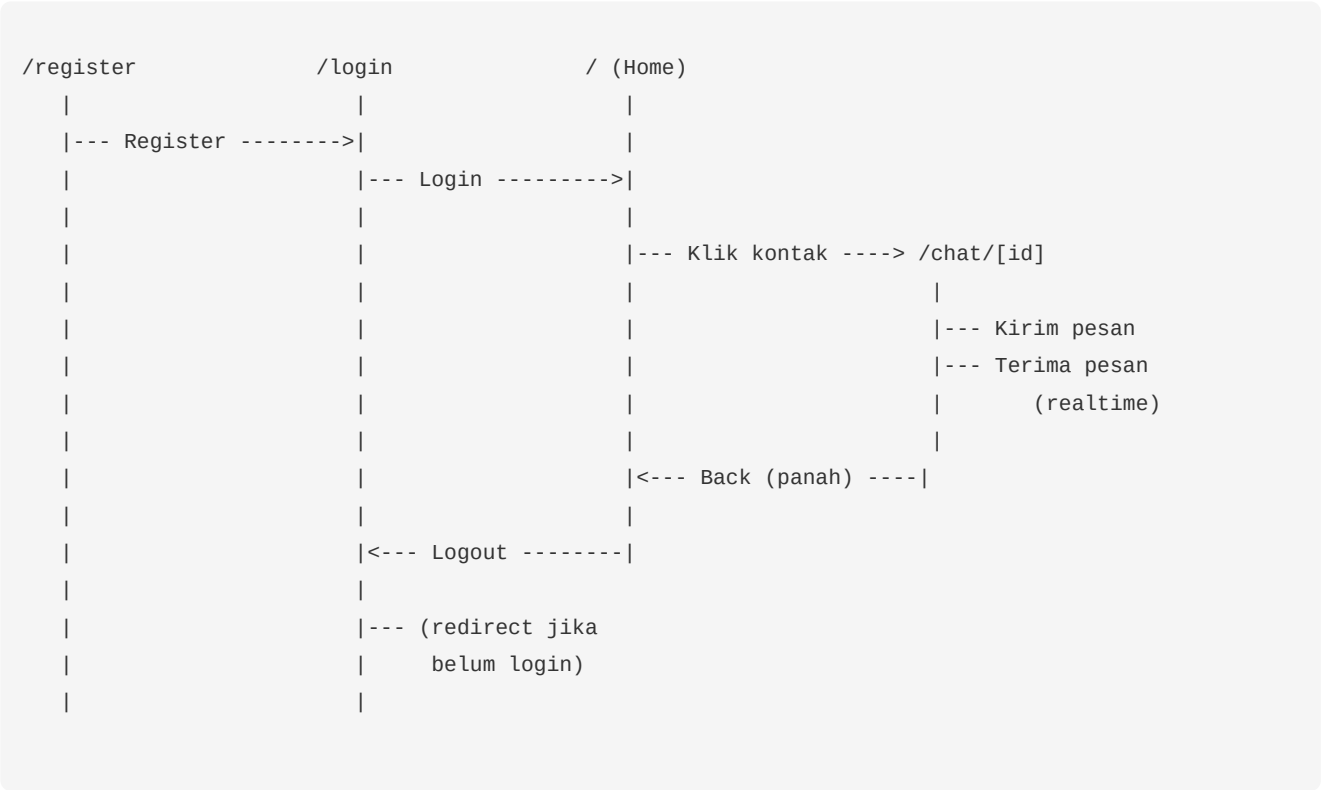
Variable	Value	Description
PORT	3001	Port Express server
DB_HOST	localhost	Host MySQL
DB_USER	root	User MySQL
DB_PASSWORD	(kosong)	Password MySQL
DB_NAME	chattingan	Nama database
JWT_SECRET	chattingan_secret_key_2024	Secret key JWT
APP_BASE_URL	http://localhost:3001	Base URL backend
TIMEZONE	Asia/Jakarta	Timezone default

Port yang Digunakan

Service	Port
Frontend (Next.js)	3000
Backend (Express)	3001
Database (MySQL)	3306

6. Alur Navigasi Halaman

Berikut adalah alur navigasi antar halaman dalam aplikasi:



Penjelasan tiap halaman:

- /register: Form registrasi user baru (name, phone, password)
- /login: Form login (phone, password) dengan redirect ke halaman utama
- / (Home): Daftar kontak yang bisa di-chat, dengan status online
- /chat/[id]: Room chat 2 arah dengan real-time messaging

7. Alur Inisialisasi Aplikasi

7.1 Frontend (_app.js)

Urutan inisialisasi saat aplikasi frontend dijalankan:

```
1. AuthProvider (React Context)
|-- state: { user, loading }
|-- useEffect on mount:
    - checkAuth() -> GET /api/auth/me
    - Jika valid: set user data
    - Jika tidak: set loading=false (redirect ke /login)

2. ChatProvider (React Context)
|-- state: { users, messages, onlineUsers }
|-- useEffect ketika user tersedia:
    - initSocket() -> konek WebSocket ke BE
    - fetchUsers() -> GET /api/messages/users
    - Listen event "online_users" dari Socket.IO

3. ConfigProvider (Ant Design)
|-- theme: colorPrimary=#1677ff, borderRadius=8

4. MainLayout (Wrapper)
|-- div container dengan styling global
|-- Render Component sesuai route
```


7.2 Backend (chattingan.js)

Urutan inisialisasi saat aplikasi backend dijalankan:

```
1. Load environment variables (.env)
  |-- dotenv.config()

2. Inisialisasi Express App (app.js)
  |-- CORS: origin=http://localhost:3000 (FE)
  |-- JSON body parser
  |-- Cookie parser
  |-- Morgan HTTP logger
  |-- Route mounting:
      /api/v1/auth      -> routes/v1/auth.js
      /api/v1/messages -> routes/v1/messages.js

3. Koneksi Database (Sequelize)
  |-- MySQL connection ke localhost
  |-- Test koneksi via .authenticate()
  |-- Sync models:
      - users table (create if not exists)
      - messages table (create if not exists)

4. WebSocket Server (Socket.IO)
  |-- Attach ke HTTP server
  |-- CORS: origin=http://localhost:3000
  |-- Middleware: JWT authentication
  |-- Event handlers:
      - connection:
          * Extract user dari JWT
          * Join room "user:{userId}"
          * Broadcast daftar online users
      - disconnect:
          * Broadcast daftar online users (updated)

5. Start HTTP Server
  |-- Listen on port 3001
  |-- Log: "Server running on port 3001"
```

8. Format Response API

Semua response API menggunakan format standar JSON.

Response Sukses

```
{
  "status_code": 200,
  "data": {
    "id": 1,
    "name": "John Doe",
    "phone": "08123456789"
  }
}
```

Response Error

```
{
  "status_code": 400,
  "timestamp": "2024-01-01 12:00:00.000",
  "error_title": "Validasi Gagal",
  "error_message": "Nomor telepon sudah terdaftar",
  "path": "http://localhost:3001/api/v1/auth/register"
}
```

Status Code yang Digunakan

Status Code	Description
200	OK - Request berhasil
201	Created - Resource berhasil dibuat
400	Bad Request - Input tidak valid
401	Unauthorized - Token tidak valid/kadaluarsa
404	Not Found - Resource tidak ditemukan
405	Method Not Allowed - HTTP method tidak sesuai
500	Internal Server Error - Error sistem

9. Flow Diagram (Textual)

Berikut ringkasan flow untuk setiap skenario utama dalam bentuk diagram tekstual:

Registrasi

[REGISTRASI]

```
Browser -> POST /api/auth/register (name, phone, password)
  -> [BE] Validasi input
  -> [BE] Hash password (bcrypt)
  -> [BE] Simpan user ke database
  -> [BE] Generate JWT token
  -> [FE] Set cookie + redirect ke /
```

Login

[LOGIN]

```
Browser -> POST /api/auth/login (phone, password)
  -> [BE] Cari user by phone
  -> [BE] Verifikasi password (bcrypt.compare)
  -> [BE] Generate JWT token
  -> [FE] Set cookie + redirect ke /
```

Chat 2 Arah

[CHAT 2 ARAH]

```
User A -> Klik User B di daftar chat
  -> GET /api/messages/[userId] (ambil riwayat)
  -> Tampilkan pesan-pesan sebelumnya
  -> Ketik pesan + Enter
  -> POST /api/messages/[userId] (message)
  -> [BE] Simpan ke database
  -> [BE] Emit "new_message" ke User B via Socket.IO
  -> User B langsung lihat pesan (real-time)
```

Logout

[LOGOUT]

```
User -> Klik tombol "Keluar"  
      -> GET /api/auth/logout  
      -> [BE] Hapus cookie token  
      -> Redirect ke /login
```

Chattingan

Copyright © 2026 - All Rights Reserved

Dokumentasi ini bersifat rahasia dan hanya untuk internal tim.